
Proof-Carrying Neurosymbolic Supervision: State, Logic-Governed Decisions, and Test-Evidence Reuse

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Coding agents repeatedly reconstruct repository state, regenerate plans, and re-
2 run validation, yet shorter prompts or cached passes can omit dependencies and
3 weaken assurance. We present an obligation-centered architecture that maintains
4 content-addressed semantic state separately from operational task state and durable
5 evidence. A formalization layer maps code, intent, and applicable constraints to
6 typed obligations while exposing unsupported or lossy translations. A supervisor
7 uses these obligations to select deterministic analysis, compositional reasoning,
8 proof search, bounded synthesis, or an explicitly authorized neural residual. Candi-
9 date edits are checked against current source; test and proof results are admitted
10 only for their declared statements, environments, and trust profiles. Accepted
11 transitions update durable roots and invalidate affected planning and verification
12 state. The design includes incremental proof-unit sealing and staged fixture-aware
13 test reuse, semantic-world memory, and procedural synthesis. We specify separate
14 tests of preservation, progress, and end-to-end cost, including matched raw-context
15 baselines, cold validation oracles, and positive authorization cases. **[RESULTS:**
16 **insert the frozen task population, useful completion rate, net measured cost,**
17 **uncertainty intervals, and false-admission results.]** The contribution is a method
18 for governing repository evolution with explicit evidence, not a claim that hashing,
19 model confidence, or every available solver proves arbitrary Python correct.

20 1 Problem and contribution

21 The cost of an agent is not only the tokens used to generate a patch. It includes rediscovering
22 relevant code, reconstructing assumptions, choosing tests, recovering interrupted work, and deciding
23 whether previous evidence still applies. Repository agents and automated testing studies make the
24 quality of the agent–tool boundary central to useful repair [8, 11]. Here, that boundary is a persistent
25 state-and-obligation representation rather than an unstructured conversation history.

26 Our proposed method has three linked contributions. First, it compiles source and domain intent into
27 identity-bound semantic evidence and explicit unresolved obligations. Second, it makes planning,
28 counterexample-guided repair, and procedural synthesis consumers of those obligations, retaining
29 neural models only for unresolved proposals. Third, it separates test observations, proof statements,
30 reuse decisions, and accepted state transitions, enabling incremental work without silently upgrading
31 evidence. Semantic context reduction is one measurable consequence of this loop, not its sole
32 purpose.

33 The prototype contains source-present components and incompletely qualified integration paths. This
34 working paper distinguishes those from methods specified in active workstreams and from results
35 actually measured. Source presence, branch landing, operational consumption, passing qualification,

Table 1: The semantic root, accepted-transition manifest, and mutable operational projection must not be conflated.

State space	Authority and purpose	What it does not establish
Semantic state G(F)	Semantic compiler: program facts, contracts, dependency closure, approximation, source references.	Task completion, a permission grant, or all-language behavioral correctness.
Operational state O	Supervisor: goals, tasks, plan epochs, leases, attempts, outcomes, route and acceptance decisions.	A new meaning for a semantic object or proof correctness by bookkeeping.
Durable artifacts B	Store: canonical bytes, candidate indexes, proof forests, WAL and current-root CAS.	Truth, fresh applicability, or authorization merely from persistence.

and policy promotion are different statuses. The final artifact must freeze the exact multi-repository source forest and the methods it exercises. [D1, A1–A5, N1–N14]

2 State and evidence are typed, separate authorities

2.1 The persistent state model

Let F be a frozen repository forest, including source trees, declared overlays, dependency pins, and relevant environment identities. The incremental semantic index extracts stable symbols, semantic versions, typed relationships, and limitations. A separate semantic-state producer seals the finalized index into immutable facts, links, capsules, and environment bindings. It supplies pure selection and freshness operations; it does not execute tests, operate a task queue, or decide that a candidate is accepted. [D1]

2.2 Claims carry provenance and applicability

The domain-neutral IR kernel separates declarations, runs, results, and receipts. A declaration does not acquire a different semantic identity because a model or solver ran. Canonical identities bind a schema/domain and canonicalization profile; operational roots can reference declarations and their histories without inserting attempts, clocks, or leases into semantic content. Legal, Security, and Intent families share these protocols but do not share verdict authority or ontology. [N4]

$$e = (\text{claim, subjects, assumptions, scope, dependencies,} \\ \text{translation profile, checker result, environment, policy, provenance}). \quad (1)$$

Admission checks integrity, source binding, current applicability, required checker evidence, and policy. These dimensions are not one scalar confidence score. A proof of an encoded property is not permission to perform an action; a signed statement is not independently observed execution; a candidate stored under a CID is not admitted knowledge. The hypothesis is that exact identities and explicit conditions enable reliable reuse—not that cryptographic hashes establish universal semantic equivalence.

2.3 Incrementality with a visible unknown frontier

Previous and current semantic states drive additive invalidation and test/proof selection. Fresh exact dependency capsules may substitute for unchanged source under a declared profile; edited targets, stale records, and opaque or insufficient analyses require exact source or broader validation. Dynamic dispatch, reflection, fixtures, plugins, external services, and generated code can prevent a complete dependency closure. Unknown edges widen the frontier rather than disappear from it. [D1; W1–W2]

Table 2: Formal tools are selected by the obligation and evidence profile; not every tool runs on every task.

Formal mechanism	Question it answers	Required qualification
Abstract interpretation	Which values, effects, or paths remain possible?	Sound over-approximation for the supported fragment; opacity elsewhere.
Assume-guarantee	Do producer guarantees meet consumer assumptions?	Typed implication, nonvacuity, and valid induction for cycles.
SMT / interpolation	Is an encoded obligation satisfied or refuted; what boundary fact is useful?	Exact theory, formulas, bounds, solver, result checks and translation profile.
Tactician / Hammer / proof assistant	Which premises, tactics, or proof candidates close a hole?	Candidate search separated from reconstruction or kernel checking.
Temporal / deontic / cognitive / F-logic views	Which state, action, obligation, or knowledge relation is being expressed?	A property-preserving encoding and an available compatible backend.
ZK relation / attestation	Which private witness satisfies which public statement?	Circuit/program, public inputs, keys and verifier; no inferred execution claim.

64 3 Formalization: preserve the property before proving it

65 3.1 Domain intent is not execution authority

66 A high-level objective is first a candidate specification. Intent IR expresses goals, actions, resources,
67 and effects; Legal IR contributes applicable normative constraints; Security IR contributes principal,
68 resource, and effect invariants. Their shared IR kernel supplies canonical claims and evidence
69 references, not an automatic equivalence between domains. Reviewed formalization and source
70 mappings turn a candidate into bounded obligations; ambiguity, missing coverage, and unsupported
71 modalities remain explicit. [N4–N7]

72 The attested authorization design queries a manifest-bound proof corpus, checks applicability and
73 revocation, composes obligations, and produces an exact-context decision. Correctness and authoriza-
74 tion are separate gates. The inspected reference enforcement layer supports one-time consumption
75 and context rechecks, but does not itself connect to real tools; the documented receipt-consumption
76 rollout defaults off/shadow. Thus the paper claims an interface and mechanism only at the deployment
77 stage actually exercised. [N7, N11]

78 3.2 Loss-aware translation and solver-state limits

79 A target formula is useful only for claims preserved by its translation. For example, a legacy projection
80 removes temporal/deontic operators and approximates a binary temporal formula by conjunction.
81 A proof of that projection cannot prove the original obligation. The admitted route must reject
82 unsupported loss or bind an independently justified preservation relation. Translation validation
83 checks the particular transformation rather than trusting every conversion [4]. [N8]

84 The inspected incremental SMT wrapper maintains named assertions and replay identity but rebuilds
85 a Z3 solver per check; that path supplies no exported proof object. It therefore supports assertion-
86 level incrementality, not an established retained-clause speedup. Solver diversity, same-solver core
87 rechecking, native proof reconstruction, and kernel checking must be reported separately. [D2]

88 The actual component checker lowers a producer guarantee G and consumer assumption A to $G \wedge$
89 $\neg A$; SAT supplies a countermodel, UNSAT discharges the encoded implication, and unsupported
90 clauses remain uncovered. The bounded QF_LIA CEGAR adapter searches a Boolean abstraction,
91 concretizes an error trace, and refines only spurious traces using validated predicates. A feasible trace
92 refutes the model; incomplete search is not a proof. These results become reasoned plan obligations
93 and localized repair inputs. [N12–N13]

94 4 Planning and state repair before neural execution

95 4.1 Objective compilation and plan admission

96 The direct-objective workstream replaces manual prompt elaboration with a bounded compiler:
97 authorized input and repository scan yield goals, subgoals, task dependencies, assumptions, permitted
98 effects, exact outputs, and acceptance obligations. Deterministic templates cover known task families.
99 A model may propose a residual decomposition, but the result must pass schema, dependency, scope,
100 resource, and acceptance-coverage checks. An inferred specification is still a proposal until admitted;
101 a model must not lower the objective or remove a required test to make its own plan succeed. [W2;
102 DOEP]

103 Planning maintains an obligation frontier. Resolved obligations reference current evidence; unre-
104 solved obligations carry a reason and a permissible next route. Hard authority, safety, and effect
105 conditions are checked before ranking by cost. The intended routing policy selects the cheapest
106 adequate route, not a ritual invocation of every theorem prover.

107 current evidence / applicable procedure
108 → exact dependency and static analysis
109 → contract, abstract-state and solver checks
110 → bounded deterministic repair or synthesis
111 → explicitly authorized residual model
112 → independent validation / review / honest insufficiency

113 4.2 A routing gate is not a proof kernel

114 The pre-implementation kernel can defer unavailable capabilities, choose a unique analytical can-
115 didate, abstain on ambiguous candidates, or authorize a named neural residual using resolved task/
116 forest-bound receipts. It does not run a model, prove a candidate, or by itself complete a task. A
117 Boolean “closes claim” on a candidate requires independent downstream justification. Its local zero
118 provider-hook count is not a measurement of saved model calls across the workflow. [A1, N1]

119 Integration is part of the method. The real caller must bind the current plan, diagnostic view and
120 obligation graph to the same task and forest as its evidence. A valid authorized residual must progress
121 while a minimally altered invalid request fails. Measuring only rejection would reward a deny-all
122 agent rather than useful neurosymbolic control.

123 4.3 Event-driven reassessment and bounded convergence

124 Repository, dependency, policy, proof, fixture, capability, and owner events invalidate only the
125 affected causal/semantic frontier when that frontier is complete. The supervisor retains accepted
126 unaffected subplans and emits a PlanDelta rather than rebuilding an entire prompt. Refill creates
127 only tasks needed for explicit gaps or counterexamples, subject to plan epochs, generation limits,
128 repeated-candidate detection, and authority. Unknown provider effects enter reconciliation rather
129 than blind retry. [W2; DOEP]

130 Queue exhaustion is not success. Completion requires current evidence covering every mandatory
131 acceptance obligation and no unresolved required consumer. Bounded termination is also not con-
132 vergence to a solution: budget exhaustion, nonconvergence, unavailable tools, and human decisions
133 remain separate terminal outcomes. Incoming world-model self-hosting is qualified only when route
134 and acceptance records show actual consumption of state, not merely sidecar logging. [W1–W2]

135 5 Repair, context compilation, and procedural synthesis

136 5.1 Localize and preserve the intended change

137 A repair begins with a state-bound failed obligation, observation, or counterexample—not a free-form
138 request to “fix everything.” Static dependency analysis and abstract interpretation bound the affected
139 region. Assume–guarantee checks expose unmet component interfaces; validated unsat cores or
140 interpolants can identify a compact missing premise. CEGAR refines an abstraction after a spurious

141 counterexample; CEGIS refines candidate implementations against accumulated counterexamples
142 [3, 5]. Conflating these loops would obscure whether the program or its abstraction changed.
143 Candidate plans, patch sketches, and CodeEditPackets remain non-authoritative. The currently
144 represented path checks preimages and scope, applies a Git patch in an isolated fenced worktree,
145 rescans semantic state, and runs selected plus required fallback validation. General tokenizer-alias IR
146 editing and exact CST/AST relinking remain separate PCSM extensions until their actual resolver,
147 renderer, round-trip and adversarial checks are frozen. A source map alone is not a proof of semantic
148 preservation.

149 5.2 Context minimizes unresolved information, not obligations

150 The semantic compiler owns semantic ContextPack identity and sufficiency; the runtime compiles
151 provider-facing ContextCapsules. The goal, authority, scope, and acceptance core is non-truncatable.
152 Optional evidence is chosen under byte/token budgets; exclusions have exact expansion references
153 and reason codes. Required target source, caveats, tests, and proof obligations cannot be removed
154 to hit a compression target. Parent-bound delta capsules carry new counterexamples or dependency
155 facts while retaining the invariant core. Stable prefixes and provider cache reuse remain bound to
156 model/tokenizer/policy identities. [D1; ASEH, PCSM]

157 A model request names the residual question, admitted evidence, missing information, budget,
158 expected response schema, and independent acceptance test. Failed candidates expand only implicated
159 source or contracts when possible. This adapts abstraction refinement to context selection without
160 claiming that every context heuristic is a sound program abstraction.

161 5.3 Compile repeated successful work into bounded operators

162 ProcedureCegis already implements a bounded synthesis state machine. Its source order is existing
163 verified procedure, built-in template, anti-unified pattern, declarative/constraint candidate, model
164 sketch, and human candidate. Candidate and counterexample-set identities prevent repeated evaluation
165 of the same pair. The synthesizer neither executes nor certifies nor promotes its output. [A2]

166 The incoming memory method mines independently admitted trajectories, preserves failed episodes,
167 infers applicability conditions, and validates anti-unified procedures on held-out families. Each
168 procedure has typed parameters, effects, branch/loop bounds, residual holes, rollback, and required
169 validation. Execution must recheck current applicability and authority. Learned call-target, event-
170 delta, inverse-trace, and repair-delta models rank constrained candidates or abstain; their predicted
171 transitions become accepted state only after observation or verification. [W2, W4]

172 Architecture refactoring applies the same loop to module boundaries and state ownership: SCC-based
173 candidates, compatibility contracts, source-preserving edits, differential traces, tests/proofs, and
174 rollback. A smaller dependency graph, fewer files, or lower context burden only ranks candidates; it
175 does not establish equivalence. [W4]

176 6 Test certificates and sound conditions for reuse

177 6.1 Specify exactly what is certified

178 A test certificate must name its statement, source/dependency roots, runner and environment, fixture
179 inputs, result semantics, and checker/trust profile. “Verified” is not a single universal evidence class.
180 A manifest can be complete while its child observations depend on a trusted runner; a circuit can
181 correctly verify signatures while proving nothing about honest execution by their signer. [K1, A5]

182 The source-present Groth16 adapter is gated by explicit opt-in and a compiled Rust backend, with
183 BN254 and versioned circuit references. Its inspected derivation helper accepts only propositional
184 atoms, facts, and one-antecedent implications, using forward chaining. This is not the full temporal/
185 deontic system and not a CPython/pytest execution model. Versioned wrapper code, setup availability,
186 and attestation metadata must not be mistaken for a qualified circuit/key ceremony or measured
187 prover performance. [N9–N10]

Table 3: Zero-knowledge can hide witnesses; it does not enlarge the meaning of the proved relation.

Evidence class	Established relation	Remaining assumption or limit
Integrity / membership	These exact bytes or leaves belong to this commitment.	Correctness and execution are not implied.
Signed runner observation	A named authorized signer asserts the bound result.	Runner integrity and observation remain trusted.
Formal proof	The encoded property follows in the declared logic/profile.	Translation adequacy, premises and checker TCB.
Aggregation / seal	Declared child checks and manifest/transition rules hold.	No stronger meaning than the verified child relations.
Direct execution proof	The specified machine/program produces the specified result.	Actual modeled runtime, I/O oracles, inputs and bounds.

6.2 Fixture-aware, phase-aware applicability

The PCTDD workstream separates collection seed, fixture-definition closure, materialized fixture-instance commitment, pre-call execution key, and post-execution setup/call/teardown evidence. It binds parametrization, plugins/hooks, confset, autouse fixtures, DI adapters, external snapshots, clocks/RNGs, interpreter, policy, and relevant hardware. Arbitrary Python object serialization is not a sound dependency model. Setup/call reuse may not silently skip required resource effects or teardown; a teardown failure prevents a whole-item pass. Opaque dependencies force execution. [W3]

6.3 Exact reuse and cross-tree reuse are distinct

A full-tree cache key changes on an unrelated edit. Broader reuse therefore needs a current dependency projection and a noninterference/applicability witness, not just removal of the tree from a key. If a deterministic observation under profile θ is a function $f\theta$ of a complete projection πD , then $\pi D(s)=\pi D(s')$ implies equal observations. This conditional argument is elementary; proving that πD includes all relevant inputs and effects is the hard obligation. Runtime traces alone cannot show that no unobserved dependency exists. [W3; Appendix I]

7 Incremental proof state and durable acceptance

7.1 Completeness is independent of membership

Verification is partitioned into source-addressed proof units with stable logical identities and complete applicability/trust keys. A requirement manifest declares every mandatory unit; a proof forest binds the evidence for those units in deterministic categories. Merkle membership proves inclusion, not that the collector included all required tests. Selection completeness and authorized removal of obligations are separately checked. Delta seals name the accepted parent and exact reused, replaced, added and removed units; uncertainty forces broader verification or a full checkpoint. [A5, K1]

7.2 Prepared artifacts cannot publish themselves

The durable store persists canonical blocks and candidate evidence; the supervisor runtime decides whether evidence satisfies current obligations. The small persistence adapter and actual pointer CAS are separate surfaces. Current-seal publication checks repository/branch identity, expected parent and generation. Local locks, atomic filesystem publication, and WAL recovery do not constitute global distributed consensus or a transaction spanning Git, the task database, and remote services. Crash recovery must explicitly reconcile those boundaries. [K1–K4]

218 candidate state and complete requirement manifest
 219 → parallel prepare: canonicalize / hash / verify / build leaves
 220 → recheck exact parent, scope, evidence and completeness
 221 → persist children before referencing manifests
 222 → one expected-parent publication barrier
 223 → reconcile task disposition and publish accepted transition

224 Parallel canonicalization, hashing, independent proof checks, and Merkle levels are staged PCTDD
 225 optimizations. They must preserve sequential bytes, CID profiles, category ordering, roots, and
 226 dispositions. Chunk-manifest identities are distinct from a raw-file SHA-256 digest. Hash memos
 227 need immutable content/profile bindings; a path or modification time alone is insufficient. Required
 228 proof-pending states may continue local preparation but remain non-mergeable until mandatory
 229 evidence is admitted. [W3]

230 7.3 Acceptance invariant and remaining trusted base

$$\begin{aligned} \text{Publish}(s') \Rightarrow & \text{CurrentParent}(s) \wedge \text{Authorized}(a, s) \\ & \wedge \text{Scoped}(a) \wedge \text{CompleteManifest}(q, s') \\ & \wedge \text{EvidenceAdmitted}(q, s') \wedge \text{CAS}(s, s'). \end{aligned} \quad (2)$$

231 This is a design invariant to qualify on the actual call chain, not a theorem that all current code
 232 enforces it. A conditional preservation argument is possible: if the initial accepted state meets the
 233 selected artifact policy, every admitted transition checks that policy on exact inputs, and publication
 234 is exclusive, induction preserves that policy along the accepted lineage. It says nothing stronger than
 235 the checked policy and does not prove all physical side effects exactly once. Appendix I names the
 236 assumptions and proof obligations.

237 7.4 Multi-agent and world-state extension

238 Sibling supervisors exchange typed requests, observations, and receipts, not writable access to
 239 each other's state. Mergeable observational indexes can nominate dependencies; exclusive claims,
 240 fences, policy, publication, and completion remain transactional. Accepted world roots can compose
 241 semantic, program-trace, logical-dataset, procedure, proof, and environment roots. Cyclic logical
 242 graphs use stable IDs and snapshot manifests rather than literal cyclic content links. Required-mode
 243 operation must exhibit pre-state consumption, accepted transition evidence, and post-state publication.
 244 [W1–W2; DOEPI]

245 8 Evaluation: useful progress, preservation, and cost

246 8.1 Primary comparison and independent oracles

247 Freeze a task population, model revisions, exact repository forest, tools, policies, budgets, and
 248 acceptance criteria before running paired conditions. The core A/B comparison changes only raw/
 249 lexical versus semantic context; later arms add routing, incremental verification/reuse, and governed
 250 lifecycle. These combined arms measure package effects, not the isolated contribution of each
 251 mechanism. Orthogonal ablations must isolate translation checking, symbolic routing, context
 252 selection, proof reuse, parallel preparation, and procedures. [ASEH; PCPR]

253 8.2 Populations and controls

254 Use exact pre-fix historical tasks, held-out task/repository families, and controlled adversarial cases.
 255 Hide target patches and acceptance oracles from proposal generation; generated tests may supplement
 256 but not silently replace the oracle. Include valid authorized actions as well as invalid ones. Seed
 257 lost modalities, inconsistent assumptions, removed tests, missing fixture inputs, stale maps/keys,
 258 changed verification keys, unknown provider outcomes, and crashes around publication. Keep
 259 failures, timeouts, abstentions and unavailable routes visible.

260 Measure successful accepted tasks per fixed budget, false acceptance and false denial, conditional
 261 solve rate after dispatch, time-to-first-valid-task, event-to-replan latency, task churn, manual repair,

Table 4: Pair safety endpoints with useful-progress endpoints. Zero unsafe actions is trivial for a deny-all system.

Study	Matched comparison	Primary correctness / progress outcome
Context	Raw/retrieved source vs semantic packs; same model/checks.	Completed accepted tasks; missed regressions; tokens and fallback.
Symbolic control	Model-first vs obligation-driven routing.	Correct deterministic closures and authorized residual dispatch, not just blocked calls.
Repair / synthesis	Existing template/CEGIS vs model proposal under same scope.	Independent hidden/full validation; specification preservation; rejected candidates.
Test / proof reuse	Cold execution vs admitted reuse; dependency-changing attacks.	False reuse; cold-oracle agreement; fixture lifecycle; coverage.
Sealing / state	Sequential vs parallel; crashes and stale writers.	Identical roots/dispositions, accepted lineage, recovery and useful continuation.

Table 5: Repository-reported preliminary evidence; not experiments rerun during this paper revision. [A4–A5]

Existing report	Bounded observation	Cannot establish
Semantic context harness	40 controlled tasks; 58.90% median estimated context reduction; no model receipts.	Live inference savings, general repair accuracy, or useful provider dispatch.
Incremental sealer	40 transitions; 43.6161% reuse; 43.0539% estimated compute reduction.	Measured prover speedup; GPU cost; proof that arbitrary tests ran.
Current source inspection	Typed state, routing, formalization, bounded synthesis, and storage mechanisms.	A single fully integrated deployment, unpushed worktree behavior, or passing runtime qualification.

and each evidence class. Instrument actual provider boundaries: a gate that makes no model call internally does not establish that the workflow saved a model call. Warm-cache and cold-start conditions require separate results.

8.3 Economic and statistical reporting

Report actual input/output/cached tokens, model calls, provider charges, CPU/GPU time, memory, storage, fixture setup/call/teardown, proof generation/checking, recovery, and human time. Currency conversions require a stated tariff; unavailable values are not zero. Measure end-to-end elapsed time directly rather than summing overlapping parallel durations. Report amortization and the break-even number of repeated tasks.

Use paired differences, medians, confidence intervals, failure counts with denominators, and stratification by dynamic-feature and task class. A finite zero-failure count is not universal safety. Preregister a quality/noninferiority margin and minimum useful completion threshold; efficiency promotion requires both. The final results and exact tested forest remain **[TO BE FILLED]**.

275 9 Evidence limits, related work, and conclusion

276 9.1 What the existing artifacts establish

277 The reviewed standalone repository heads and the consumer’s dependency pins differ. A final claim
278 must bind the actual loaded source forest, including overlays, native binaries, solver/circuit profiles
279 and active rollout settings. Source-present intent authorization and a Groth16 wrapper must not
280 be described as always-on enforcement or a validated universal prover. The preceding integration
281 findings belong in the author audit until resolved or reported as limitations.

282 9.2 Relationship to established methods

283 Proof-carrying code attaches checkable policy evidence to untrusted code [1]; abstract interpretation
284 and CEGAR reason over approximations and refine them with counterexamples [2, 3]. Translation
285 validation checks a produced transformation, and CEGIS searches bounded candidates against
286 counterexamples [4, 5]. Equality saturation represents alternatives under rewrite relations [6]. These
287 are foundations, not newly invented algorithms in this work.

288 The intended difference is composition at the agent’s operational boundary: persistent semantic
289 state, explicit obligations, residual model authorization, phase-aware test evidence, and accepted-
290 transition publication. Build-system research separates scheduling from rebuilding [7]; our reuse
291 design additionally exposes statement, fixture, trust and authority conditions. Coding agents and
292 test-generation benchmarks [8, 11], premise-retrieval and hammer systems [9, 10], and prompt
293 compression [12] provide relevant baselines. We do not claim that this is the first neurosymbolic
294 agent or that combining these techniques automatically improves performance.

295 9.3 Limitations and accountable self-improvement

296 Python reflection, native code, external state, concurrency, incomplete specifications, and unsound
297 translations limit what can be proved. A sound checker cannot repair an incorrectly formalized
298 requirement. Default-deny behavior can mask broken positive paths. A self-improving supervisor
299 must not edit its own protected acceptance specification, keys, validators or promotion policy under
300 ordinary task authority. Every learned update or compiled procedure requires an independent holdout
301 and rollback gate.

302 Important model use in implementation, formalization, candidate generation, experiments and
303 manuscript preparation must be disclosed accurately. The final claims remain conditional on the
304 frozen evidence and tested deployment. Our thesis is that persistent obligations can displace repeated
305 neural reasoning and validation while preserving a declared acceptance standard; the paired evaluation
306 must determine whether the total benefit exceeds the added analysis and proof cost.

References

- [1] George C. Necula. Proof-carrying code. POPL, 1997, pp. 106–119. DOI: 10.1145/263699.263712.
- [2] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. POPL, 1977, pp. 238–252.
- [3] Edmund M. Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. CAV, 2000, pp. 154–169. DOI: 10.1007/10722167_15.
- [4] Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. TACAS, 1998, pp. 151–166.
- [5] Armando Solar-Lezama. Program Synthesis by Sketching. PhD dissertation, University of California, Berkeley, 2008.
- [6] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. POPL, 2021. DOI: 10.1145/3434304.
- [7] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. ICFP, 2018. Expanded as Build systems à la carte: theory and practice, Journal of Functional Programming 30, 2020.
- [8] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. NeurIPS 37, 2024.
- [9] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J. Prenger, and Animashree Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. NeurIPS 36, 2023.
- [10] Albert Qiaochu Jiang, Wenda Li, Szymon Tworkowski, Konrad Czechowski, Tomasz Odrzygóźdź, Piotr Miłoś, Yuhuai Wu, and Mateja Jamnik. Thor: Wielding hammers to integrate language models and automated theorem provers. NeurIPS 35, 2022.
- [11] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. SWT-Bench: Testing and validating real-world bug-fixes with code agents. NeurIPS 37, 2024.
- [12] Zhuoshi Pan et al. LLMingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. Findings of ACL, 2024, pp. 963–981. DOI: 10.18653/v1/2024.findings-acl.57.

A One complete state to repair to certificate cycle

Scenario: a repository operation writes a record before checking a caller's permitted scope. The authorized requirement is that a denied request produces no persistent write. R0, P0 and T0 below are illustrative labels, not recorded benchmark CIDs. The example exercises the planned integration rather than claiming an observed end-to-end run.

Table 6: Illustrative state-to-repair-to-certificate cycle; labels are schematic.

Step	Artifact and acceptance condition
1. Capture state	Freeze source tree R0, recursive dependencies, environment E0, policy P0 and semantic root S0. Extract the call/effect edges for authorize and write_record; keep any opaque store behavior visible.
2. State the obligation	O0: denied(request, P0) implies persistent_state_after = persistent_state_before for the declared observation profile. Confirm this is an authorized requirement; do not infer it only from existing tests.
3. Obtain a counterexample	A denied request under fixture instance F0 produces a write. Bind the failing call trace, fixture-definition and instance commitments, runner profile, setup/call/teardown outcomes and exact source.
4. Plan a bounded repair	Localize the write-before-check ordering. Admit a plan that changes only this operation and its justified dependencies. The candidate must preserve authorized writes, error behavior, finalization and public API semantics.
5. Choose a reasoning route	Try an existing checked guard-before-effect template. Use abstract effects or an encoded transition property to discharge what the model supports. If ordering or callbacks remain ambiguous, expand exact source or request a bounded model residual—not an unrestricted rewrite.
6. Validate the candidate	Apply to a fenced disposable worktree R1, rescan to S1 and invalidate the affected contracts/tests/proof units. Execute relevant tests and full fallback where required. A fake store test does not prove production database semantics.
7. Reuse narrowly	A proof unit outside the complete affected closure may be reused only after current binding and evidence checks. A policy or fixture change invalidates every dependent unit even if the test function text is unchanged.
8. Seal and publish	Admit the current requirement manifest, persist verified children and a parent-bound delta seal, then compare-and-swap the expected pointer/generation. Reconcile the operational task only after matching publication and validation receipts.
9. Learn without self-approval	Store the accepted transition and rejected candidates. Repeated independently validated instances may nominate a guard-before-effect procedure; synthesis, certification and promotion remain separate.

A.1 Counterfactual checks

Change F0, P0, the interpreter, a plugin, teardown behavior or an external snapshot without editing the test body: the key must change or reuse must be rejected. Deliver a duplicate event or late worker completion: the current state must not regress. Crash after immutable persistence but before pointer

345 CAS: recovery must not invent completion. Supply an empty manifest while O0 is required: reject
 346 incompleteness.

347 The example’s proof claim is limited to the selected model and observations. A successful unit test, a
 348 checked transition formula and an attested runner result remain different artifacts with different trust
 349 assumptions.

350 B A typed evidence model and verification obligations

351 B.1 Evidence record

352 Evidence = (claim, subject_roots, statement_or_observation, scope,
 353 assumptions, dependency_roots, translation_receipts,
 354 checker_profile, checker_result, environment, policy,
 355 producer_or_signer, invalidation_bindings, provenance)
 356 Admit(e, q) = Integrity(e) $\wedge$ SchemaValid(e) $\wedge$ Applicable(e, q)
 357 $\wedge$ RequiredCheckPasses(e, q) $\wedge$ PolicyAllows(e, q)

358 This is a proposed unifying notation, not a replacement for existing schema authorities. Integrity
 359 checks exact bytes; applicability checks freshness and scope; required checking evaluates the selected
 360 statement; policy checks execution/acceptance authority. The conjunction cannot be satisfied by a
 361 model’s confidence score or a single untyped success flag. A verified proof may originate from an
 362 untrusted model: the checking rule, not the proposal’s reputation, establishes the new claim.

Table 7: Formal mechanisms and their required evidence.

Mechanism	Representation and required limit
Abstract interpretation	Bind transfer semantics, abstract domain and widening/narrowing strategy. Unsupported operations over-approximate or become opaque. Do not call the entire Python front end sound solely because one abstract domain is well defined. [D3]
Assume-guarantee	Bind producer/consumer clauses, source map, theory, invariants and normal/exceptional observations. Check assumption consistency and cyclic closure; expose uncovered prose clauses. [D4]
Incremental SMT	Bind full formula/assertion roots, declarations, theory, translator, solver configuration and replay manifest. The current wrapper rebuilds its solver; benchmark any future retained-state implementation separately. [D2]
Interpolation and refinement	Validate implication, contradiction and shared vocabulary; preserve unsupported/timeout outcomes. CEGAR refines abstractions, while CEGIS refines bounded candidates against accumulated counterexamples. [D5]
Tactician / Hammer / proof assistants	Premise and tactic predictions are candidates. Invoke the actual available backend and native reconstruction or checker required by the profile; no fake bridge, universal provider or cross-logic soundness assumption. [D7; W2, W4]
Translation / e-graph extraction	Freeze source/target languages, supported observations, rule trust and proof replay. Equi-satisfiability, refinement and observational equivalence are different claims. Opaque interface names are not implementation evidence. [D6]

363 B.2 Targeted source-level checks before the final freeze

364 Add tests for duplicate or colliding named SMT assertion/assumption identifiers, exact formula-root
365 binding in receipts, stale replay fingerprints, unsupported terms and rejection of unvalidated results.
366 Exercise the real caller admission path—not only a synthetic result constructor. These are review
367 obligations, not a claim that a runtime exploit was demonstrated in this audit.

368 C Fixture definitions, injected instances and reuse eligibility

369 This appendix preserves the parallel sealing/TDD directive’s staged protocol. Earlier identity and
370 trace infrastructure exists, but no new qualification of this entire protocol is asserted here. [P8; W3]

Table 8: Staged test identities and permitted reuse.

Stage	Required bindings	Decision allowed
0 · collection seed	Test node, parameters, marks, collection config, source/AST identity, static closure, plugins and adapters.	Candidate lookup only; not a passing outcome.
1 · fixture-definition closure	Fixture names, definitions, scopes, autouse/usefixtures, parameterization, transitive dependencies, conftest, hooks, setup/teardown/finalizers, purity/effect policy.	Whole-item pre-setup reuse only under a qualified closed policy.
2 · fixture-instance commitment	Resolved parameter values and reviewed injected commitments; filesystem/database snapshot; service image and response corpus; transaction/lease, clock/RNG and setup outcome.	Post-setup eligibility; opaque or mutable unbound values force execution.
3 · execution key	Test/source/fixture closure, instance roots, runtime and dependency lock, toolchain, relevant hardware, effect/network policy, trace completeness, statement/circuit/key profile.	Re-admit a prior receipt or execute the call. No optimistic test pass.
4 · phase receipt	Setup, call and teardown outcomes, failures, effects, observed dependency trace, runner identity, final acceptance and signature/proof relation.	Fresh observation and evidence for later reuse.

371 C.1 Reviewed dependency-injection adapters

372 Admit canonical primitives, bounded typed records, exact file/resource CIDs, versioned configuration
373 allowlists, immutable database snapshots, reviewed service fakes or committed response corpora
374 through explicit adapters. Do not serialize arbitrary callbacks, sockets, secrets, unrestricted object
375 graphs or live databases as if they were complete immutable inputs. Bind adapter version, privacy
376 policy and any absence or non-determinism. [W3]

377 C.2 Preserve lifecycle effects

378 Separate reuse of fixture definitions, fixture instances and test-call results. Module/session-scoped
379 resources, parametrized instances and xdist worker ownership require scope-aware scheduling. If
380 setup ran, required teardown/finalizers still run even when a call result is reusable. Teardown failure
381 prevents a whole-item pass. Scheduling must preserve resource isolation and observed ordering
382 requirements, not simply distribute tests with matching names. [W3]

383 A trace records what was observed on one execution. It does not prove that a branch not taken
384 could not access another dependency. Exact reuse across states needs a closed behavioral boundary;

otherwise widen the closure and execute. Keep cold executions and mutation cohorts to test the reuse policy independently.

C.3 Proof batches, parallel preparation and the inner loop

C.3.1 Certificate and batch statement

A batch statement binds the test inventory/requirement manifest, child execution keys and admitted outcomes, count, ordering, duplicate policy and aggregate root. A cryptographic profile must bind its public inputs, any private commitments, circuit or program, verification key, setup origin, verifier version and intended relation. A proof of signature verification still relies on the signer’s execution claim; it is not direct proof of the runner. Zero-knowledge is justified only by a defined privacy need and a reviewed statement. [W3]

Reject unknown circuits or key origins, test-only keys on production allowlists, silent key generation, simulated required units, and outer metadata not bound to the verified statement. Keep proving keys and witness material outside public proof-store APIs. If recursive verification is unavailable, represent a manifest of individually verified leaves rather than inventing a recursive proof. [K1; W3]

C.3.2 Parallelism that preserves identity

Table 9: Parallel preparation and serial publication obligations.

Preparation job	Required safety property
Read / parse / canonicalize	Use an exact immutable input or verified overlay. Exclude symlink escapes, unstable reads and mutable pathname assumptions; canonicalize once under a fixed profile.
Hash memo	Key exact content and relevant canonicalization/codec/hash/source-mode policy. Reverify candidates; mtime or a fast digest alone is not the final SHA/CID authority.
Hash / verify workers	Parallelize independent records/files/proof checks with bounded CPU, memory, process/GPU usage, cancellation and deadlines.
Merkle construction	Sort leaves, build categories and same-level nodes concurrently, preserve normative order. Memoize unchanged verified branches and recompute paths from changed leaves.
Serial barrier	Recheck completeness and current identities, persist children, then use the accepted expected parent/generation. Worker completion order cannot change bytes or disposition.

C.3.3 Development-loop states

The fast loop combines source capture, semantic invalidation, selection, cache-candidate lookup, fixture-aware execution, candidate repair and certificate production. Proof generation can proceed asynchronously inside the product only when the intermediate state remains explicitly non-mergeable. Queue starvation, cancellation, proof failure, missing keys or unknown execution must preserve honest status. [W3]

The directive’s rollout is bootstrap → shadow_hash → shadow_reuse → shadow_proof → protected → required. These gates are not interchangeable with world-model shadow-read semantics. Measure call/setup/teardown time, proof latency, batch verification, hash/serialization throughput, cache corruption rejection, serial barrier time and total cost. Performance targets are acceptance goals—not results. [W3]

411 **D Semantic objects, projection indexes and logical datasets**

412 **D.1 Semantic objects and relations**

413 Wrap existing domain-state CIDs in adjacent versioned semantic-object envelopes containing domain,
414 schema, canonicalization profile, provenance, source CIDs, optional declared policy salt and evidence
415 class. Do not alter the existing source-semantic root to include embeddings, prompts, actors, leases,
416 wall time or transition history. Pure semantic deltas remain distinct from operational execution
417 records. [W1, W2]

418 Relations are scoped claims such as equivalent, refines, entails, contradicts, compatible_with and
419 supersedes. Each binds left/right objects, scope, assumptions, environment and theory/policy, proof/
420 evidence references, limitations and invalidation dependencies. “Similar” belongs to a retrieval
421 projection, not the authoritative semantic relation vocabulary. Exact CID equality establishes byte
422 identity, not universal semantic equivalence. [W1]

423 **D.2 Hybrid retrieval without authority leakage**

424 Projection records bind their semantic object, projection kind, model, tokenizer, preprocessor, dimen-
425 sion, metric, normalization, vector identity and privacy policy. Rebuildable index manifests bind
426 generations and model/projection compatibility. Lexical, vector, LSH/Hamming, graph or neural
427 search may nominate candidates. The resolver must retrieve exact object bytes, verify current root
428 membership and recheck freshness, environment, obligations and policy before use. A stale vector or
429 approximate neighbor never substitutes for an exact subject. [W1, W2]

430 The graph-to-graph or typed-attention TAGSeq/DAGSeq idea remains an optional experimental
431 projection over typed program neighborhoods. It must be compared with exact lookup, rules, ordinary
432 graph features and embedding-only baselines. Canonical symbols and typed source maps remain
433 authoritative even if the learned projection changes. [W2]

434 **D.3 Logical data versus physical layout**

435 LogicalDatasetState and PhysicalDatasetLayout must not collapse into one identity. Under a fixed
436 logical schema, key/order policy and provenance, changing Parquet row groups, compression or
437 shard packaging should preserve logical content identity while changing layout identity. Set, multiset
438 and ordered semantics must be explicit; duplicate and key-conflict behavior cannot be inferred from
439 file order. [W1]

440 Transformations bind exact inputs, transformation specification, environment/toolchain, candidate
441 outputs and validation evidence. Merges use a closed declared semantics—append, keyed three-way,
442 ordered, set/multiset, schema evolution or conflict. Distributed workers may prepare shards and
443 candidate layouts, but only accepted logical state and validated provenance may enter a world root.
444 Legacy or mock dataset managers cannot silently remain production semantic authorities. [W1, W2]

445 **D.4 VFS bridge**

446 A successful canonical VFS/worktree mutation emits a durable post-commit receipt and semantic
447 outbox record. Rescanning, invalidation and acceptance occur through the existing semantic and
448 supervisor authorities. Filesystem success alone is not task success; recovery handles an outbox that
449 exists before semantic acceptance, and all side effects remain attributable to exact operations. [W1,
450 W2]

451 D.5 Frames, events, transitions and learned continuation models

452 D.5.1 State schema and abstraction profile

453 ProgramStateSnapshot:
454 repository/environment/program-graph roots; source location; stack;
455 selected locals or shapes; abstract heap/alias summary; path condition;
456 exception state; obligations; capabilities/effects; external observations;
457 capture profile; abstraction profile; confidence and limitations
458 ProgramTransition:
459 before-state; action; event; after-state or delta; conditions;
460 observation/validation evidence; accepted/rejected/candidate status

461 Profiles declare stack depth, value precision, heap slicing, allocation identity, timestamp treatment,
462 exception handling and the observations preserved or erased. Absence is typed: a trace adapter that
463 cannot capture an external service is not a complete program-state observer. Logical execution graphs
464 may cycle without creating literal cyclic content-addressed blocks. [W2]

465 D.5.2 Three distinct inference problems

Table 10: Program-prediction tasks and candidate semantics.

Task	Candidate semantics and validation
Next call	Use lexical scope, imports/exports, nominal/structural types, alias/points-to information, inheritance, dispatch tables, callbacks, plugins and observed traces to build a conservative set. Rank existing targets; keep unknown-dynamic probability and abstention.
Next event / abstract state delta	Predict call/return/raise/handle/branch/await/yield/resume/I/O/mutation/contract/check events with a declared state abstraction. Validate by replay, execution, contracts or the appropriate formal backend.
Inverse trace	Return multiple predecessor hypotheses, missing calls/inputs and replay recipes from a partial failure. Non-invertibility and observational ambiguity are expected; do not train it as a unique true reversal.

466 Symbolic pruning may exclude a candidate only through an admitted type, path, alias, effect, temporal,
467 resource, authority, exception or contract constraint. Uncertainty is not exclusion. A repair-delta
468 model proposes bounded changes to canonical nodes/edges; an exact resolver and source-preserving
469 renderer still need independently validated source correspondence. [W2]

470 D.5.3 Dataset and promotion protocol

471 Freeze capture/abstraction profiles, provenance, source-family splits, candidate-set construction and
472 targets before evaluation. Separate observed from synthetic/static-derived examples and distinguish
473 valid alternatives from wrong predictions. Report candidate recall before ranking, top-k, calibration,
474 abstention, unsupported cases, replay validity and downstream repair value. Model scores do not
475 become facts or required certificate evidence. [W2]

476 The predecessor proof-grounded learning campaign and a successor’s uncompleted work cannot share
477 a positive-result label. Preserve the earlier reported no-go until a distinct frozen learned artifact and
478 independent evaluation establish a new claim. This review does not establish new learned weights, a
479 tokenizer or a universal interlingua. [P6, P11; W2]

480 **E Behavior-preserving remodularization, not cosmetic compression**

481 **E.1 Model the observations to preserve**

482 Track function, class, block and module identity at multiple levels: exact source/CST, semantic imple-
483 mentation, binding, contract, effects, mutable state, dependencies, public interface and compatibility.
484 Formatting, relocation, renaming, aliasing and semantic change need different dispositions. Public
485 behavior includes return values, exceptions, mutation, initialization order, import effects, serialization/
486 pickle names, introspection, plugin discovery, cancellation, timeouts, resources and documented API
487 surfaces—not just existing happy-path tests. [A3; W4]

488 **E.2 Analyze before partitioning**

489 Build a typed architecture graph of calls, reads/writes, authority owners, providers, interfaces, receipt
490 producers, stores, tests and proofs. Identify SCCs, cycles, import-time ordering and shared state. A
491 partition candidate must preserve a coherent state owner and an explicit boundary contract; do not
492 break a dependency cycle by hiding it behind opaque callbacks or duplicated state. Similarity and
493 semantic-entropy scores nominate candidates but prove neither duplication nor dead code. [A3; W4]

494 **E.3 Bounded transformation and validation**

495 architecture finding → partition candidate → boundary contracts
496 → scoped operator + reversible migration plan
497 → isolated worktree + CST/AST-aware transform
498 → translation / differential / effect / public-surface checks
499 → selected and full-fallback tests + supported proofs
500 → independent admission + exact rollback + updated state

501 Reuse a closed operator grammar: extract module/interface/pure function, move state to its owner,
502 introduce typed adapters or dependency inversion, quarantine legacy/simulation paths, replace
503 ambiguous outcomes, deprecate a public symbol or remove confirmed dead code after the relevant
504 gate. Operators bind preconditions, scope, effects, migration, consumer coverage and rollback.
505 Unknown dynamic reachability is not proof of deadness. [A3; W4]

506 Use property-based, metamorphic, mutation and differential executions over declared observables,
507 plus type/effect checks and supported solver or native proof obligations. Tactician/Hammer routes
508 can try to discharge obligations; bounded synthesis or a small model may propose a residual. A test
509 suite cannot alone establish universal observational equivalence of arbitrary Python. Contradictory
510 authoritative premises require conflict resolution, not proof by inconsistency to authorize a rewrite.
511 [W4]

512 **E.4 Fixed-point control and memory**

513 Persist accepted and rejected partitions, failed applicability, counterexamples and migration costs.
514 Recompute only affected graph regions; stop when no admissible beneficial candidate remains or a
515 budget is exhausted. Public API, mutable-state, security, legal, payment, key, release-authority and
516 wire-protocol changes retain their human-approval ceilings. Learned benefit or lower context cost
517 cannot raise those ceilings. [A3; W4]

518 The public architecture checkpoint establishes inventory and ArchitectureIR foundations. The broader
519 SCC/partition/CST/remodularization program remains separately tracked against the author's local
520 worktree. [B1; W4]

521 **F Rollout, event federation and verified self-hosting**

522 **F.1 Reconcile predecessor programs**

523 The semantic-world and program-world directives explicitly require extending an accepted predeces-
524 sor or creating a sealed successor—not two stores, scanners, schedulers or root authorities. Apply
525 the same rule to existing procedure, architecture, proof reuse, sealer, governor and meta-controller

526 campaigns. Read the live owner-controlled board and dependency seals before adding tasks; old plan
527 status text does not override current code or active local work. [W1–W4]

528 **F.2 Distinct rollout state machines**

Table 11: Distinct rollout state machines.

Program	Meaning of progression
World / program-world	Bootstrap → shadow_write → shadow_read → guarded → required. In the program-world directive, shadow_read computes hypothetical decisions without influencing behavior. Guarded allows only bounded consumption under independent checks. Required makes pre-root consumption and post-root acceptance part of task completion.
Parallel sealing / TDD	Bootstrap → shadow_hash → shadow_reuse → shadow_proof → protected → required. Require sequential parity and zero unsafe divergence on the fixed adversarial cohort before protective gates can depend on the new path.
Refactor / procedures	Separate candidate generation, independent qualification, shadow use and policy-authorized promotion. No worker changes its own assurance threshold or rollout mode.

529 **F.3 Mandatory per-task receipts in required mode**

530 A task consumes an exact source forest, semantic/program state, obligation set, route decision and
531 context plan. It produces candidate artifacts, observations, current validation, admitted or rejected
532 transition, and a matching post-root/completion record. A deployment that writes optional semantic
533 metadata while continuing to decide from raw prompts has not demonstrated required self-hosting.
534 [W2, W3]

535 **F.4 Concurrency and persistence**

536 The operational database is accessed through the current typed owner boundary; event and outbox
537 records, revisions, claims and fences must agree. Delivery can be at least once while logical operations
538 are idempotent. Causal graphs and affected-frontier wakeups avoid duplicate reconstruction, but
539 retrieval projections and optional history stores cannot create readiness, leases or acceptance. [P3;
540 W2]

541 CRDT-like merging can be useful for observational knowledge with explicit conflict semantics. It
542 must not merge exclusive leases, authorization, current roots, irreversible effects or completion by
543 optimistic set union. Speculative agents use isolated branches and candidate proof/premise prefetch;
544 only independently admitted outputs cross the serial publication boundary. Remote storage and
545 libp2p/MCP++ transport require their own authentication/capability and fault tests. [W1, W2]

546 **F.5 Typed service parity**

547 Python, CLI and MCP are projections of the same operation contracts and authority rules. An
548 envelope identity is not the semantic payload identity. Delegated capabilities authorize an operation
549 but do not establish its mathematical correctness. Keep secrets and raw private traces outside prompt,
550 receipt and public-storage surfaces; ordinary imports must not launch daemons or installers. [D1;
551 W1–W4]

552 G Experiments and acceptance gates for the landing work

Table 12: Experimental cohorts and required fault checks.

Cohort / ablation	Required measurements and failure checks
State and selection	Cold/incremental byte-root parity; deletion/rename/config/fixture/native/dynamic changes; selected-vs-full regressions; ambiguity and raw-source fallback. Empty denominators stay not-applicable.
Planning and repair	Same tasks, models, budgets and acceptance criteria. Count calls by route, deterministic closures, proof holes, repaired/unsolved/escalated tasks, duplicate retries and human interventions.
Certificate reuse	Cold oracle executions with changed fixtures, instances, finalizers, plugins, policies, external snapshots and runner versions. Report each attempted false reuse, not only aggregate hit rate.
Parallel sealer	Equal bytes/roots/dispositions at multiple worker counts; malformed blocks, cache corruption, interrupted writes and stale-parent races. Separate prepare, verify, persist, CAS, CPU/GPU and memory costs.
Procedures / world models	Held-out repository-family and time-based splits; candidate recall, applicability, replay, calibration/abstention, negative memory and transfer. Synthesis/training cannot inspect held-out acceptance outputs.
Architecture refactoring	Frozen before/after tasks; API/import/exception/effect/serialization behavior; dependency cone and context burden; migration, validation, rollback and subsequent maintenance cost.
Recovery and federation	Duplicate/reordered events, worker crashes, expiry/fencing, unknown external effects, store unavailable, network partition, cross-repository dirty overlays and conflicting gitlinks.

553 G.1 Cost and uncertainty

554 T_{total} = elapsed wall time from task admission to terminal disposition
555 Stage report = capture, compile, retrieve, model, setup, test, teardown,
556 prove, verify, persist, retry, recover and human waits
557 Money_total = billed provider cost + measured infrastructure allocation
558 Resource report = CPU time, GPU time, peak memory, bytes, storage growth

559 Overlapping parallel stage times are not additive wall time. Do not add seconds, tokens and dollars as
560 if they were one physical quantity. A policy may optimize a declared weighted utility, but the paper
561 must also report the separate observable units. Amortize cold indexing, proof setup, training and
562 procedure compilation over an explicit number of tasks; include break-even behavior and workloads
563 where the method is slower.

564 G.2 Hard gates and honest dispositions

565 Require no observed unsafe admission, stale publication, forged certificate acceptance or false reuse in
566 the frozen adversarial suite, while reporting its finite size and uncertainty. Quality must not be traded
567 away to meet a speed target. An unavailable backend, an incomplete proof or baseline-compatible
568 failing test remains visible. A non-promotion can be the correct campaign result.

569 The final anonymous bundle needs a source forest, install/lock and capability manifests, frozen
570 task population/splits, model settings, test/prover commands, raw receipts, analysis code, failure

571 taxonomy, trusted-base manifest and replay instructions. A reviewer must be able to regenerate each
572 table without private author state.

573 H Early supervisor programs and their scientific role

574 The chat contains more than the four saved world/TDD/refactoring directives. The earlier programs
575 below are retained explicitly. They are reconciled with existing implementation owners rather than
576 treated as commands to launch duplicate boards. A handoff prompt specifies intended work; it is not
577 evidence that a supervisor was launched or that a worktree landed.

Table 13: Earlier supervisor programs and their qualification requirements.

Program	Method preserved	Qualification needed
ASEH · efficiency and state hardening	Paired end-to-end telemetry; deterministic-first routing; shared contexts; one state-transition authority; removal of duplicate writable paths.	Actual provider calls, success and cost at matched budgets; event/recovery and warm/cold baselines.
DOEP · direct objective and event planning	High-level idea → typed objective → goal/task DAG; event-driven PlanDelta and bounded refill; sibling clients; nonconvergence and authority controls.	Valid input reaches useful tasks; changed state invalidates affected work; no runaway objectives or queue-empty success.
PCPR · platform qualification	Truthful unavailable/simulation statuses, canonical path cutovers, real backend qualification, immutable dependencies and release manifest.	One loaded integrated forest, active rollout, clean installation and current evidence; not three unrelated main-head results.
PCSM · semantic minification	Canonical semantic IR versus local projection; M1 source-linked edits, M2 declared abstraction, M3 advisory learning; tokenizer costs and CEGAR expansion.	Separate slicing/alias/linking ablations; exact target source; round-trip and stale-map tests; no universal AST linker inferred from a patch schema.

578 H.1 Dependency order, not a new campaign taxonomy

579 First qualify state and applicability, then the symbolic route and real positive model-dispatch path,
580 then repair and test-certificate admission. Only after those seams work should a combined run
581 attribute gains to automatic refill, minification, procedures or learned world state. Existing gates and
582 protected validation owners remain authoritative.

583 H.2 Independent status dimensions

584 location: main | branch | local_overlay | specification_only
585 implementation: present | partial | incompatible | missing
586 integration: called | adapter_only | optional | unknown
587 validation: not_run | reported | reproduced | failed | blocked
588 rollout: off | shadow | guarded | required
589 paper claim: exact mechanism + population + scope + limitations

590 A branch may already be an ancestor of main; an importable module may never be consumed; a
591 passing negative fixture may never exercise a valid path. Update each dimension independently. Do
592 not replace pending outcomes with the expected landing date.

593 H.3 World state, tests, and refactoring are parts of one loop

Table 14: World-state, test, and refactoring methods in the supervision loop.

Directive	Mechanisms retained	Connection to supervision
W1 · semantic-addressed world	Semantic objects and scoped relations; canonical profiles; projection/index generations; exact resolution; logical datasets vs layout; VFS outbox and world-root CAS.	Accepted pre/action/post transitions become reusable state; similarity can nominate but never substitute for exact resolution.
W2 · program world	Frames/stacks/heaps, capture/abstraction profiles, typed traces; static candidates and symbolic pruning; next-call, next-event, inverse-trace and graph-delta residuals.	Obligations and counterexamples determine route, context, subgoals and tasks; learned output stays candidate-tier until validated.
W3 · parallel sealing / TDD	Hash memo and parallel preparation; deterministic forests; fixture-definition/instance and DI keys; whole-item vs call reuse; setup/call/teardown; xdist; async certificates.	Avoid repeated work only when current inputs, effects, evidence and publication checks permit it; cold execution is the reference.
W4 · semantic refactoring	Multi-level identities; authority/state graphs; SCC partitions; boundary contracts; CST-aware codemods; initialization, API, pickle and plugin compatibility.	Repair/refactor waves preserve observations under declared profiles; verified episodes support bounded procedures, negative memory and fixed-point reanalysis.

594 The four saved directives collectively specify 126 top-level sections, including operational and report-
595 ing requirements. Their previous complete heading index remains in the companion reconciliation
596 handoff; it has not been replaced by this method summary. The latest revision retains Appendices
597 A–G and adds exact source-level qualification issues rather than discarding the previously discussed
598 mechanisms.

599 H.3.1 Self-hosting must be observable

600 In shadow-write the system computes sidecar artifacts; in shadow-read it evaluates hypothetical
601 decisions without changing authority. Guarded consumption and required self-hosting must be
602 separately approved. In required mode each eligible task consumes a verified pre-root, current
603 obligations and route/context decision, then publishes observations, validation, accepted/rejected
604 transition and completion/post-root evidence. Workers may not modify rollout, keys, protected
605 specifications or their own acceptance validator.

606 H.3.2 Learned representations remain bounded research

607 Typed-attention/TAGSeq/DAGSeq-style models are optional comparisons after static, history, lexical
608 and embedding baselines. Program graphs may be cyclic; storage manifests are not. Predicting a next
609 event, recovering a predecessor trace and proposing a repair are different objectives with different
610 labels. Track calibration, abstention, static-candidate recall, leakage-safe repository-family/time
611 splits, and downstream acceptance. Attention weights, proximity and a generated CID are not proofs.

612 I A semantic preservation contract for every translation

613 The formal layer is not a bag of interchangeable solver adapters. For each source-to-target conversion
614 τ , record source domain, target logic/theory, vocabulary, variable/clock interpretation, assumptions,

615 unsupported constructs, omissions, source map and claimed preservation direction. Source truth may
 616 be inferred from target truth only when that implication is established for the requested property. [N4,
 617 N8]

$$\text{TargetChecker}(\tau(\varphi)) = \text{accepted} \not\equiv \text{SourceSemantics} \models \varphi. \quad (3)$$

Table 15: Logic views, their useful roles, and their explicit scope.

Source view / mechanism	Useful role	Scope to bind explicitly
Typed software IR / first-order formulas	Contracts, alias/effect relations, claim dependencies.	Supported syntax and extractor semantics; unknown behavior stays possible.
F-logic / object-frame views	Structured entities and relationships.	Actual engine/profile and translation tests; an advertised family is not an installed backend.
TDFOL / temporal-deontic view	Obligations, permissions, prohibitions and temporal constraints.	Clock/trace semantics, modal interpretation and property-preserving lowering.
DCEC / cognitive-event or event-calculus view	Actions, events, agents and knowledge/intention relations.	Domain axioms and supported modal/event fragment; no unqualified flattening to ordinary predicates.
SMT arithmetic / arrays / bit-vectors	Finite checks, satisfiability, inductive or bounded obligations.	Selected theory, bit width, bounds, result type, solver and replay identity.
Lean / Coq / ATP + reconstruction	Candidate proof search and independently checked theorems.	Proof payload, theorem/axioms, native checker version and exact reconstruction result.
ZK helper named TDFOL_v1	Finite atom/implication derivation witness for an optional backend.	It accepts P and $P \rightarrow Q$; this name does not imply full temporal, deontic or first-order semantics.

618 I.1 Concrete loss test to add

619 The inspected legacy converter maps $O(p)$ to p and a temporal operator applied to p to p ; it also
 620 approximates a binary temporal formula by conjunction. Construct distinct source states or traces
 621 in which the original modality differs while the projection agrees. The acceptance path must reject
 622 lifting a target proof to the source claim unless an appropriate preservation condition is supplied. This
 623 is a proposed semantic regression, not a newly executed repository test. [N8]

624 I.2 Nonvacuity and contradictory premises

625 Before accepting $G \Rightarrow A$ in component composition, check the intended consistency and reachability
 626 conditions. Unsatisfiable G makes the implication vacuously true; it may expose a broken specification
 627 rather than a correct component. Unknown is not inconsistent and not proved. Mutating axioms or
 628 normative policy to close a proof is a new authorized specification change, not automatic repair.

629 I.3 What can be proved about acceptance and reuse

630 I.3.1 Accepted-lineage invariant — proof sketch, not machine checked

631 Fix a statement profile Θ and a policy predicate $P\Theta$ over artifact manifests. Assume: the initial
 632 accepted root satisfies $P\Theta$; all required subject bytes and dependency references are verified; the
 633 manifest is complete under an admitted completeness rule; each accepted evidence item satisfies its
 634 designated checker relation; the sole publication operation enforces the expected parent and all gates;
 635 and protected policy/checker identities cannot change under worker authority.

Then every root reachable solely through accepted publication transitions satisfies $P\Theta$. Proof: induction on the number of successful publications. The initial case is assumed. For the induction step, the publisher checks $P\Theta$ on the exact candidate manifest and current parent before its CAS linearization; failed or stale attempts are not successors. The induction does not establish any property omitted by $P\Theta$, truth of unsupported translations, honest signed observations, or atomicity of external side effects.

642 I.3.2 Dependency-projected test reuse — proof obligation

Let $\text{Obs}\theta(t,s)$ be the relevant setup/call/teardown observation of test t in state s under a declared runtime/effect profile θ . Suppose a complete projection πD and deterministic function $f\theta$ exist such that $\text{Obs}\theta(t,s)=f\theta(\pi D(s))$ on all states admitted by the reuse policy. If $\pi D(s)=\pi D(s')$, equality of the observations follows immediately. An admitted previous observation can therefore transfer only within that profile and those assumptions.

The difficult part is establishing completeness and determinism of πD . Definitions alone miss fixture values; function coverage misses plugin, clock, network, DB and teardown effects. Two equal digests of incomplete manifests do not satisfy the premise. For effectful tests the profile must also show that required effects are replayed or observationally unnecessary. Otherwise run setup and/or the test and teardown. A cold oracle and adversarial mutations check the implementation but do not prove universal closure.

654 I.3.3 Three acceptance questions, three different answers

Behavioral claim: does candidate program/model satisfy its obligation?
 Action permission: may this actor perform this action now?
 Publication: does this candidate and its evidence become current?

Neither a proof of behavior nor a capability alone answers all three. Formalization determines the statement; authorization binds actor, tool, arguments, resources, effects, scope, time and roots; execution observations report what happened; publication chooses the accepted lineage. The reference pre-dispatch code uses an in-memory consumption ledger and fake-dispatch defaults, so durable real-tool integration requires its own adapter and restart evidence. [N7, N11]

663 I.3.4 Useful autonomy requires positive-path witnesses

Require a paired witness for each gate: a correctly authorized, well-bound case that progresses and a minimally altered case that is rejected. Include a deny-all control in safety/cost diagnostics, not as a competitive repair method. Otherwise default deferral, unavailable backends or adapter mismatches can masquerade as perfect safety and low inference cost.

668 J Native proof code, metadata, and runtime observations

669 J.1 The inspected optional Groth16 path

The high-level adapter exposes Groth16/BN254 through a lazily imported Rust FFI backend and checks an explicit enable flag. Generation defaults to circuit version 1; the version-2 TDFOL_v1 branch constructs a deterministic derivation trace and refuses a theorem outside its derivable fragment. The helper's grammar is limited to named atoms, facts, and unary-antecedent implications. A Python helper producing a trace is not by itself evidence that the deployed RICS enforces every intended constraint. [N9–N10]

The wrapper also supplies setup/ensure-setup functionality and allows a seed. Reproducible research setup and test-vector determinism must remain separate from approved operational key provenance. Existence of key files or a compiled binary is not a proof of key ceremony, circuit soundness, correct deployment, or acceptable proving cost.

680 J.2 Bind inner statements to outer claims

The inspected `verify_proof` path forwards JSON decoded from `proof_data` to the FFI verifier. Metadata and attestation views are also assembled around that object. At the consumer boundary, compare the

683 actual verified public inputs with the source, obligation, profile, environment, circuit, key and test-
684 inventory claims used for admission. Do not infer that every outer metadata field is cryptographically
685 bound merely because the inner proof verifies. This is a required boundary test; no exploitable
686 end-to-end attack was reproduced. [N9]

Table 16: Adversarial changes at the native-proof admission boundary.

Adversarial modification	Expected disposition
Change outer source/test root but keep proof_data unchanged.	Reject application claim if it differs from verified public inputs.
Use proof of committed axioms as proof a Python test executed.	Reject mismatched statement/evidence class.
Change circuit version, verification key, profile or revocation root.	Re-evaluate applicability; never silently reuse old admission.
Omit one mandatory test from an otherwise valid batch.	Reject completeness even if every included leaf verifies.
Pass call phase but fail teardown; reuse call with stale fixture values.	No whole-item pass; execute/reject as policy requires.
Use simulated artifacts, auto-generated test keys or unknown backend.	No production credit; explicit unavailable/experimental outcome.

687 J.3 Experiment still required

688 Measure native generation and verification on a named circuit and key policy, not an estimated
689 proof-shaped record. For test attestation, run a genuine runner with independent observations; for
690 direct execution claims, name the encoded runtime and external-I/O assumptions. Report prover
691 unavailable as unavailable. Recursive verification is optional; a manifest of individually checked
692 proofs is a different and acceptable mechanism when labeled correctly.

693 K Fill from one frozen, independently scored experiment

Table 17: Main comparison; isolate component effects through matched ablations, not the combined arm alone.

Arm	Accepted solved / total	False accept / deny	Tokens / cost	Elapsed / human time
A · raw + full validation	[TBD]	[TBD]	[TBD]	[TBD]
B · semantic + same validation	[TBD]	[TBD]	[TBD]	[TBD]
C · + symbolic route / reuse	[TBD]	[TBD]	[TBD]	[TBD]
D · + governed lifecycle	[TBD]	[TBD]	[TBD]	[TBD]

Table 18: Both sides of every acceptance boundary must be exercised.

Integration witness	Safe rejection	Valid progress	Evidence ID
Pre-implementation provider gate	[TBD]	[TBD]	[TBD]
Loss-aware formal translation	[TBD]	[TBD]	[TBD]
Current native theorem/circuit profile	[TBD]	[TBD]	[TBD]
Fixture/call/teardown reuse	[TBD]	[TBD]	[TBD]
Root publication after restart	[TBD]	[TBD]	[TBD]
Required world/procedure consumption	[TBD]	[TBD]	[TBD]

694 K.1 Disclosure and reproducibility

695 Language models were used for [actual implementation assistance], [candidate goals/plans/patches],
 696 [proof candidates], [data processing], and [manuscript preparation]. Record models, versions/dates,
 697 settings, prompts or permitted summaries, data exposure and independent checks. Remove unused
 698 roles rather than imply they occurred. Do not expose secrets, proprietary source, private witnesses or
 699 private reasoning logs to satisfy a reproducibility checklist.

700 K.2 Final author actions

701 Freeze all landing worktrees and loaded dependency pins; reconcile implementations with ASEH,
 702 DOEP, PCPR, PCSM and W1–W4; qualify the positive-path gate; bound each logic translation and
 703 proof statement; regenerate result tables from raw evidence; report negative/missing results; replace
 704 artifact labels with anonymous paths; remove author-only review notes and identifying links; use
 705 the official workshop LaTeX style and check the actual PDF length. The separate audit/ledger is
 706 author-only. No repository mutation, supervisor launch, native build or live experiment is implied by
 707 this document revision.

708 L How symbolic computations change the next agent action

709 L.1 Abstract state supplies candidate bounds

710 The inspected Python abstract interpreter uses a product of flat constants, integer intervals, nullness,
 711 exceptions and effects. Its immutable lattice protocol exposes order, join, meet, widening and
 712 narrowing; the adapter records local interprocedural summaries and source positions. The worklist
 713 accumulates possible states at program points. Unsupported reflection, native calls, loading and
 714 callbacks widen to opacity. A consumer may prune an impossible candidate only under a sound
 715 admitted approximation, never from a learned low score. [N14]

$$\text{Abstract state} = \text{Const} \times \text{Interval} \times \text{Nullness} \times \text{Exceptions} \times \text{Effects}. \quad (4)$$

716 L.2 Interface composition creates a precise discharge job

717 A CompositionEdge references exact producer/consumer contracts and named normal, exceptional
 718 and induction clauses. Unknown clause references are rejected before solver execution. The compo-
 719 sition graph and receipt bind the semantic root, contracts, edges, obligation outcomes, uncovered
 720 assumptions and SCCs. Its basic implication query is $G \wedge \neg A$. SAT provides a countermodel;
 721 UNSAT provides solver-backed discharge of that encoded implication. Opaque prose stays uncovered.
 722 Cycles need a typed invariant plus an independently solved closure check rather than mutually citing
 723 guarantees. [N12]

$$G \wedge \neg A \text{ is UNSAT} \Rightarrow G \Rightarrow A \quad (\text{in the encoded theory}). \quad (5)$$

724 This permits a caller to consume a dependency guarantee instead of repeatedly loading its body
 725 only when current bindings, statement coverage and the consumer assumptions hold. A contract

726 gap becomes a named repair or proof task. Consistency and reachability checks prevent treating an
727 impossible guarantee as meaningful implementation correctness.

728 **L.3 CEGAR localizes abstraction error before code repair**

729 The QF_LIA adapter searches a supplied Boolean abstraction of a control-flow graph and constructs
730 a named concrete path formula for each abstract error trace. A feasible path is retained as a
731 real counterexample. A spurious path may be refined with a validated interpolant/core, weakest
732 precondition, or reviewed predicate. Iteration, predicate, abstract-state and trace-length budgets
733 are explicit; unavailable, unknown, timeout and budget exhaustion are separate dispositions. It is a
734 refinement adapter over existing contracts, not a second unrestricted model checker. [N13]

$$A \Rightarrow I, \quad I \wedge B \text{ is UNSAT}, \quad \text{Symbols}(I) \subseteq \text{Symbols}(A) \cap \text{Symbols}(B). \quad (6)$$

735 Those are the checks required for an interpolant at the shared boundary. An unsat core identifies
736 relevant asserted clauses but is not automatically an interpolant or a kernel proof. After localization,
737 the supervisor may expand only the implicated context, request a bounded synthesis candidate, or
738 abstain. CEGIS changes candidate programs; CEGAR changes the abstraction. [D2, D5; N13]

739 **L.4 Evidence feeds control, not just a report**

740 Each result must be consumed by the actual plan/frontier or admission service: closed implication $\rightarrow$
741 reusable component contract; feasible counterexample $\rightarrow$ repair obligation; missing semantic input
742 $\rightarrow$ exact expansion; unavailable solver $\rightarrow$ deferral/fallback; complete current evidence $\rightarrow$ candidate
743 acceptance. A sidecar analysis that never affects these decisions cannot be credited with reducing
744 LLM use. Record the consumed result CID and subsequent control transition in the experiment.

745 M Composed control-loop specification

746 The following algorithm is retained from the preceding evaluated draft as an intended composition,
747 not a newly executed implementation result. It is a control-flow specification retained for comparison
748 with the final executable artifact.

749 **Algorithm 1: Intended composed supervision loop.** Input: authorized q, current source forest F,
750 policy P, budget B

- 751 1. reconcile event, task and artifact state; refresh F and derive G, O
- 752 2. if cancelled or budget exhausted: return explicit non-success
- 753 3. if every current obligation is admitted: return verify_goal_completion()
- 754 4. generate bounded task proposals; preview and separately admit ready work
- 755 5. if no task is admissible: return wait_or_review(), not goal success
- 756 6. claim task; bind lease/fence, preimages, required context and validation
- 757 7. choose deterministic, authorized residual-model, review or unavailable route
- 758 8. if review/unavailable: record blocked attempt; return to the event service
- 759 9. produce and apply candidate only through checked isolated execution
- 760 10. rescan; derive O_new from authorized rules, not the worker's pass claim
- 761 11. verify/reuse evidence; retain failed, unknown and pending obligations
- 762 12. if refinable failure: record new residual; return bounded retry to service
- 763 13. if evidence incomplete or unsafe: return defer_or_reject()
- 764 14. persist manifest; recheck current bindings; CAS the expected parent
- 765 15. if publication conflicts: return reconcile(), not completion
- 766 16. record accepted task transition and updated F, G, O; reevaluate at line 1
- 767 17. propose procedure/policy learning separately; never self-promote

768 This algorithm specifies the composition to evaluate. Passing tests of isolated stop paths does
769 not establish the complete loop; qualification must exercise useful authorized progress as well as
770 rejection.

771 **Paper checklist: completion pending**

772 The supplied workshop template includes a separate `checklist.tex`. That questionnaire was not
773 included in the uploaded template sources. This page is an explicitly marked compilation placeholder,
774 not a reproduction of the organizers' checklist and not a declaration of compliance.

775 **[TODO]** Obtain the official questionnaire, replace this file, and complete its answers against the final
776 manuscript and frozen artifact.

777 **[TODO]** Final experiment results, source snapshots, anonymous artifact access, and author-reviewed
778 LLM-use disclosures remain pending. No checklist answer is inferred from the draft.